Colônia Aurora Siger — Sistema Integrado de Energia

Atividade Integradora — FIAP, Fase 2

Nome completo	Gustavo Hiroshi Fernandes Nakahodo
RM	576337
Turma	Ciência da Computação (On-Line)
Data	11 de novembro de 2026
Repositório	https://github.com/gustavonakahodo/aurora-siger-pre-decolagem (pasta `entrega-fase2/`)

Visão geral

O sistema recebe as leituras de energia da colônia e devolve uma ação clara. Ele faz isso em quatro passos, cada um em um arquivo de `src/`: organiza os dados, analisa o balanço de energia, decide por regras e prevê a geração das próximas horas. O programa `colonia.py` encadeia os quatro e imprime um relatório. Tudo usa apenas a biblioteca padrão do Python.

O enunciado não fornece dados. O histórico, os consumos, os limites das regras e os cenários foram definidos por nós e são sintéticos. Todos os números deste relatório foram conferidos contra a execução real do programa.

1. Como os dados foram organizados

Todos os dados ficam em um único arquivo, `data/colonia.json`, separados do código. Usamos três estruturas, cada uma para o que ela faz melhor.

Listas — o histórico. Medições em sequência, em listas paralelas: a posição *i* de uma lista corresponde à posição *i* da outra.

```
vento_ms          = [4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15]
energia_eolica_kw = [10.2, 12.1, 15.3, 17.4, 20.1, 22.8, 24.7, 27.9, 30.2, 32.1, 35.4, 37.3]
```

Chave-valor — os cenários. Cada cenário é um dicionário buscado pelo nome (`cenarios["economia"]`), com a reserva, a geração medida, a previsão do clima e a lista do que está ligado.

Hierarquia — os sistemas. A colônia é uma árvore de dicionários aninhados. Um nó com `subsistemas` é um agrupamento; um nó sem é um equipamento. Cada equipamento consumidor guarda seu consumo e sua prioridade (1 = essencial, 2 = importante, 3 = não essencial).

```
Colônia
├── Sistema energético
│   ├── Geração ── Solar, Eólico
│   └── Armazenamento ── Baterias (400 kWh)
├── Suporte à vida ── Oxigênio (12 kW), Água (8 kW), Controle térmico (10 kW)
└── Habitat ── Comunicação (4), Iluminação (6), Estufa (10),
               Laboratório (12), Lazer (8)
```

A navegação é feita por caminho: `buscar_subsistema(arvore, "energia/geracao/eolico")` desce um nível por vez. Duas funções recursivas percorrem a árvore inteira: `listar_folhas` e `consumo_total`. Esta última é o elo entre estrutura e decisão: **o consumo da colônia não é um número digitado, é a soma dos equipamentos ligados**. Com os oito ligados, o total é 70 kW; a mesma função aplicada só ao ramo "Suporte à vida" devolve 30 kW.

Antes de usar um cenário, o sistema o valida: reserva fora de 0–100 %, geração negativa ou equipamento repetido na lista de ligados são recusados com uma mensagem clara, em vez de virarem uma decisão errada.

2. Regras de decisão

As regras ficam em `src/decisoes.py`. Elas olham três números — reserva, consumo e geração — e são testadas da mais grave para a mais leve; a primeira que vale define a saída.

#	Condição	Saída
1	reserva < 20 %	`CRÍTICO: manter apenas sistemas essenciais`
2	reserva < 50 % **E** consumo > 60 kW	`ALERTA: reduzir consumo — modo economia ativado`
3	reserva < 50 %	`ALERTA: reduzir consumo`
4	consumo > geração (reserva ≥ 50 %)	`ATENÇÃO: consumo acima da geração — usando a reserva`
5	nenhuma das anteriores	`NORMAL: operação normal`

A regra 3 é a regra básica do enunciado; a regra 2 é a combinação de condições com **E**. A ordem importa: como a regra 2 é mais específica, ela vem antes da 3.

Prioridade. Quando é preciso cortar, a função `planejar_cortes` desliga primeiro a prioridade menos importante (3), depois a 2, e dentro de cada uma começa pelo maior consumo. Ela para assim que o consumo cabe na geração. Na regra 1 o corte não depende do déficit: sai tudo que não é essencial, para a reserva se recuperar o mais rápido possível. A prioridade 1 (oxigênio, água, controle térmico e comunicação) nunca entra na lista — há testes automatizados que verificam isso em milhares de combinações sorteadas de reserva, geração e equipamentos.

Exemplo (cenário `economia`). Entrada: reserva = 40 %, consumo = 70 kW, geração = 40 kW. A regra 2 dispara. O sistema desliga Laboratório (12) e Lazer (8), da prioridade 3; como ainda faltam 10 kW, desliga a Estufa (10), da prioridade 2. Saída:

```
Decisão ........... ALERTA: reduzir consumo — modo economia ativado
  regra: reserva 40.0 % < 50 % E consumo 70.0 kW > 60 kW → modo economia
Desligar .......... Laboratório, Lazer, Estufa
Consumo após ...... 40.0 kW
```

3. Modelo de previsão

Usamos **regressão linear simples**: uma reta `y = a·x + b` ajustada ao histórico pelo método dos mínimos quadrados, escrita à mão em `src/previsao.py`:

```
a = Σ (x − média_x)(y − média_y) / Σ (x − média_x)²
b = média_y − a · média_x
```

São dois modelos, um para cada fonte:

Fonte	Reta ajustada	R^2	Medições
Eólica	energia = 2,499 × vento + 0,05	0,9988	12
Solar	energia = 0,0598 × irradiância + 0,16	0,9996	10

O R^2 diz quanto da variação dos dados a reta explica; perto de 1, a reta descreve bem o histórico. Com a reta eólica, **vento = 11 m/s → 27,5 kW**, o mesmo resultado esperado pelo enunciado (≈ 27). O exemplo do enunciado (`vento = [8, 10, 12]`, `energia = [20, 25, 30]`) também é um teste automatizado: a reta sai `2,5 × vento + 0` e a previsão para 11 é 27,5.

A previsão alimenta a decisão. Para a próxima janela de 6 horas, o sistema estima a geração a partir do clima previsto (arredondada a 0,1 kW, a resolução das medições), projeta como a reserva vai ficar e aplica **as mesmas regras** sobre esse futuro. Se o resultado for pior que o presente — nível mais grave, ou mesmo nível com a reserva caindo —, emite um aviso preventivo.

Limite do modelo: a reta só vale dentro da faixa observada (vento de 4 a 15 m/s, irradiância de 100 a 1000 W/m²). Uma turbina real não gera mais para sempre com mais vento: a mesma reta daria 150 kW para um vento de 60 m/s, o que não faz sentido. Por isso, quando o clima previsto está fora da faixa, o relatório mostra `CUIDADO: extrapolação` ao lado da previsão.

4. Como o sistema melhora o uso de energia

- **Evita o apagão do que importa.** Com reserva em 15 % (cenário `critico`), o sistema desliga estufa e iluminação e mantém os 34 kW essenciais. Ninguém precisa decidir sob pressão o que cortar: a ordem já está definida pelas prioridades.

- **Corta só o necessário.** No modo economia o corte para quando o consumo iguala a geração (de 70 para 40 kW). A iluminação fica ligada porque, para a geração de agora, não é preciso desligá-la.
- **Aproveita o que sobra.** No cenário `nominal` a geração (83 kW) supera o consumo (70 kW) e a saída é `SUGESTÃO: armazenar energia excedente`. Se a bateria já estiver cheia, a sugestão muda para usar o excedente em cargas não essenciais, em vez de desperdiçá-lo.
- **Age antes do problema.** No cenário `preventivo` tudo está normal agora (reserva 65 %, saldo de +5 kW). Um sistema só reativo não faria nada. Mas a previsão de clima (vento 5 m/s, irradiância 300 W/m²) dá geração de apenas 30,6 kW contra 70 kW de consumo. A saída é:

```
DECISÃO: NORMAL: operação normal
PREVENTIVO: em 6 h a reserva vai de 65.0 % a 5.9 % (situação CRÍTICO).
            Desligando já Laboratório, Lazer, Estufa, Iluminação, a reserva
            fica em 59.9 %.
```

O aviso traz o ganho em números: esperar custa 54 pontos de reserva. O mesmo acontece no cenário `economia`, em escala menor: o corte de agora resolve o presente, mas o vento vai cair, e o sistema recomenda desligar também a iluminação para terminar a janela com 29,7 % em vez de 20,7 %.

Essa é a passagem de um sistema reativo para um preditivo: a mesma lógica de regras, aplicada a dados que a regressão estimou.

5. Repositório

Código, dados, testes e este relatório estão em:

https://github.com/gustavonakahodo/aurora-siger-pre-decolagem — pasta `entrega-fase2/`

Para executar: `cd entrega-fase2 && python src/colonia.py economia`. O `README.md` da pasta traz a saída completa do programa e as instruções dos 101 testes automatizados.